

Real-Time Communication Security Module 4

Prepared by

Jesna J S

Assistant Professor CSE, TKMCE

Topics covered

1. Perfect Forward Secrecy
2. Denial-of-service / clogging protection
3. Endpoint identifier hiding
4. Live partner reassurance

Real-Time Communication Security

- **Real-Time Communication (RTC) Security** refers to the measures and protocols used to protect data, voice, and video communication that happens instantly over networks. This includes securing technologies like VoIP (Voice over IP), video conferencing, messaging apps, and live streaming services
- A **real-time protocol** is one in which the parties negotiate interactively to authenticate each other and establish a session key, in contrast to a protocol such as email in which one party prepares a message that can later be decrypted and authenticated by the intended recipient. Standards for real time public-key-based security protocols include IPsec, SSL/TLS, and SSH.

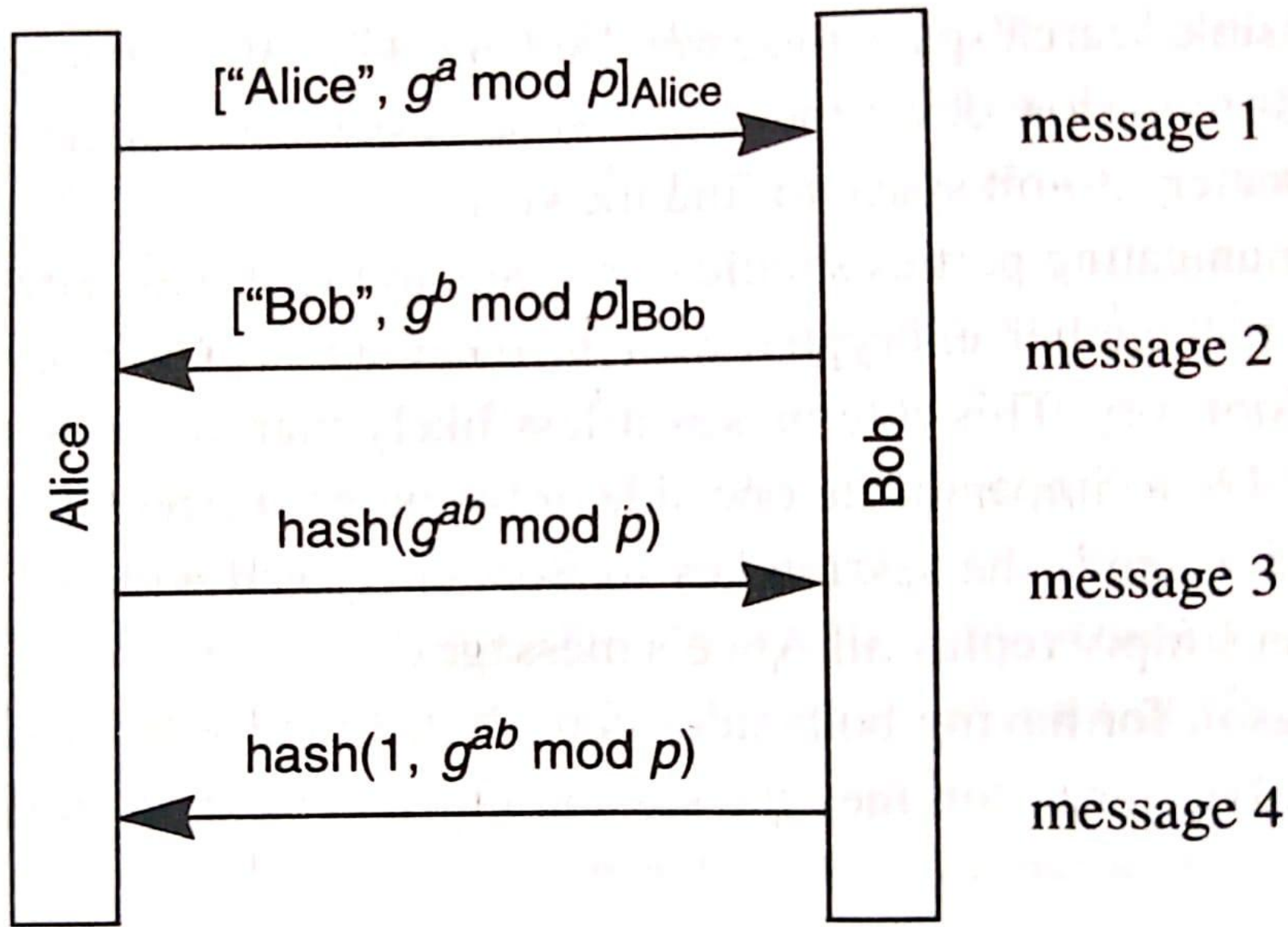
Perfect Forward Secrecy (PFS)

- It is a security property in cryptography that **ensures even if a server's long-term private key is compromised in the future, past communication sessions remain secure.**
- Without PFS:
 - ✓ An attacker records encrypted traffic today
 - ✓ Steals the server's private key later
 - ✓ Decrypts all past communications
- **With PFS, past sessions cannot be decrypted, even if keys are stolen later.**
- PFS uses **temporary (ephemeral) session keys** instead of permanent keys.
 - ✓ Each session generates a new, random secret key
 - ✓ Session keys are never stored
 - ✓ Destroyed after the session ends
- PFS is achieved using ephemeral key exchange algorithms:
 1. Diffie–Hellman Ephemeral (DHE)
 2. Elliptic Curve Diffie–Hellman Ephemeral (ECDHE)

Diffie–Hellman Ephemeral (DHE)

- Diffie–Hellman Ephemeral (DHE) is a key exchange method that provides Perfect Forward Secrecy (PFS) by **using temporary (ephemeral) keys for every session.**
- **New Diffie–Hellman keys are generated for each session**
- **Keys are discarded after the session ends**
- Even if long-term keys are compromised later, old sessions stay secure.
- Working::
 - ✓ Client and server agree on public parameters Large prime p and Generator g
 - ✓ Client chooses a **random private key a and computes public value: $g^a \bmod p$**
 - ✓ Server chooses a **random private key b and Computes public value: $g^b \bmod p$**
 - ✓ Exchange public values A and B
 - ✓ Both compute the **same shared secret: $g^{ab} \bmod p$.**
 - ✓ This shared secret is used to derive the session key

- After the session $\rightarrow$ a , b , and K are destroyed
- Why DHE Gives Perfect Forward Secrecy
 - ✓ Each session uses new private keys (a , b)
 - ✓ Long-term server private key is not used to derive session keys
 - ✓ Past encrypted traffic cannot be decrypted later



16-2. Diffie-Hellman for perfect forward secrecy using signature keys

This is a four-message exchange between Alice and Bob to securely establish a shared key while ensuring forward secrecy.

- Message 1:
 - ✓ Alice sends her identity "Alice" and her public key $g^a \bmod p$, signed with her private signature key.
 - ✓ This ensures that Bob can authenticate Alice's message.
- Message 2:
 - ✓ Bob responds with his identity "Bob" and his public key $g^b \bmod p$, signed with his own private signature key.
 - ✓ Now, Alice can authenticate Bob.
- Message 3:
 - ✓ Alice computes the shared secret key as $g^{ab} \bmod p$.
 - ✓ She sends a hash of this shared secret to Bob.
- Message 4:
 - ✓ Bob verifies the received hash and responds with another hash, incorporating a constant value (1) along with the shared key

- **Ephemeral Keys:** The values **a** and **b** are unique for each session, ensuring new keys are generated every time.
- The **ephemeral private keys** (a,b) and the **session keys** $(g^{ab} \bmod p)$ are what must be **discarded** after each session to ensure **Perfect Forward Secrecy (PFS)**.
- The ephemeral private keys $(a,b) \rightarrow$ Must be discarded after each session.
- The computed session key $(g^{ab} \bmod p) \rightarrow$ Used temporarily, then discarded.
- Any stored session logs $\rightarrow$ Secure systems ensure session keys are not reused.

Why PFS is Important for RTC

- **Protects Past Conversations:** Even if an attacker compromises a server or steals an encryption key, they cannot decrypt old calls or messages.
- **Enhances Privacy:** Users can communicate without fear of mass surveillance or data breaches exposing their past communications.
- **Mitigates Key Compromise Risks:** Traditional encryption without PFS can be vulnerable if a long-term private key is leaked.

Denial-of-service/Clogging Protection

- Denial-of-Service (DoS) attacks in real-time communication (RTC) target services like VoIP, video conferencing, and messaging by **overwhelming servers with excessive requests, making them unavailable to legitimate users.**
- Clogging attacks are a type of **resource exhaustion attack**, where an attacker sends a large number of connection requests, forcing the system to spend computational resources on fake traffic.
- To **mitigate** these threats, various protection mechanisms like **cookies and computational puzzles** are used.

Threats to RTC Systems

1. Denial-of-Service (DoS) and Distributed DoS (DDoS) in RTC
2. Clogging Attacks in RTC

Denial-of-Service (DoS) and Distributed DoS (DDoS) in RTC

- A Denial-of-Service (DoS) attack refers to a malicious attempt to disrupt a real-time communication session by **overwhelming the server with excessive traffic from a single source**, while a Distributed DoS (*DDoS*) *attack utilizes multiple compromised devices to simultaneously flood the server with traffic*, effectively preventing legitimate users from accessing the RTC service.
- **Impact on RTC Systems**
 - ✓ **Dropped VoIP Calls** – Users cannot establish or maintain voice/video calls.
 - ✓ **High Latency & Packet Loss** – Audio/video streams become distorted or freeze.
 - ✓ **Server Overload** – Excessive fake call requests or signaling traffic crashes the system.
 - ✓ **Resource Exhaustion** – Bandwidth, CPU, and memory get consumed by fake traffic.

Clogging Attacks

- A **clogging attack** is a type of **Denial of Service (DoS) attack** that aims to disrupt real-time communication by **overwhelming a system with excessive or malicious traffic**, making it difficult or impossible for legitimate users to communicate effectively.
- The attacker floods the **network or a specific service** with fake or excessive requests, causing network congestion or consuming excessive processing power.
- In real-time communication (VoIP, video conferencing, instant messaging), this results in:
 - Delays (latency)
 - Packet loss
 - Jitter (inconsistent audio/video quality)
 - Dropped calls or messages

- RTP Flooding: In **Real-time Transport Protocol (RTP)**, attackers inject excessive RTP packets, consuming bandwidth and disrupting media streams.
- To **mitigate** these threats, **cookies and computational puzzles** are commonly used to prevent **unauthorized or excessive connection attempts**.

Cookies for DoS Protection

- Cookies help prevent **fake session requests** by ensuring that only **legitimate clients** can establish a connection.
- Instead of **immediately allocating resources**, the server sends a **cookie (random value)** that the client must return to continue the process.
- **How Stateless Cookies Work**
 1. Client sends a request (e.g., VoIP call setup, WebRTC connection).
 2. Server responds with a cookie (encrypted value derived from the request).
 3. Client must return the exact cookie in the next request.
 4. Server verifies the cookie before proceeding with the connection.

- The server **does NOT store session data** until the client **proves legitimacy**, reducing the risk of clogging attacks.
- **Example: SIP Cookie-Based Protection**
 - ✓ A VoIP server receives thousands of SIP INVITE requests.
 - ✓ Instead of immediately processing them, it sends a **stateless cookie challenge**.
 - ✓ Only clients that return **valid cookies** are allowed to proceed.
 - ✓ Attackers **fail to respond correctly**, and their requests are **discarded**.
- Prevents memory exhaustion by avoiding unnecessary session allocations.
- Stops automated bots from overloading the RTC system.

The Problem: SYN Flood DoS Attack

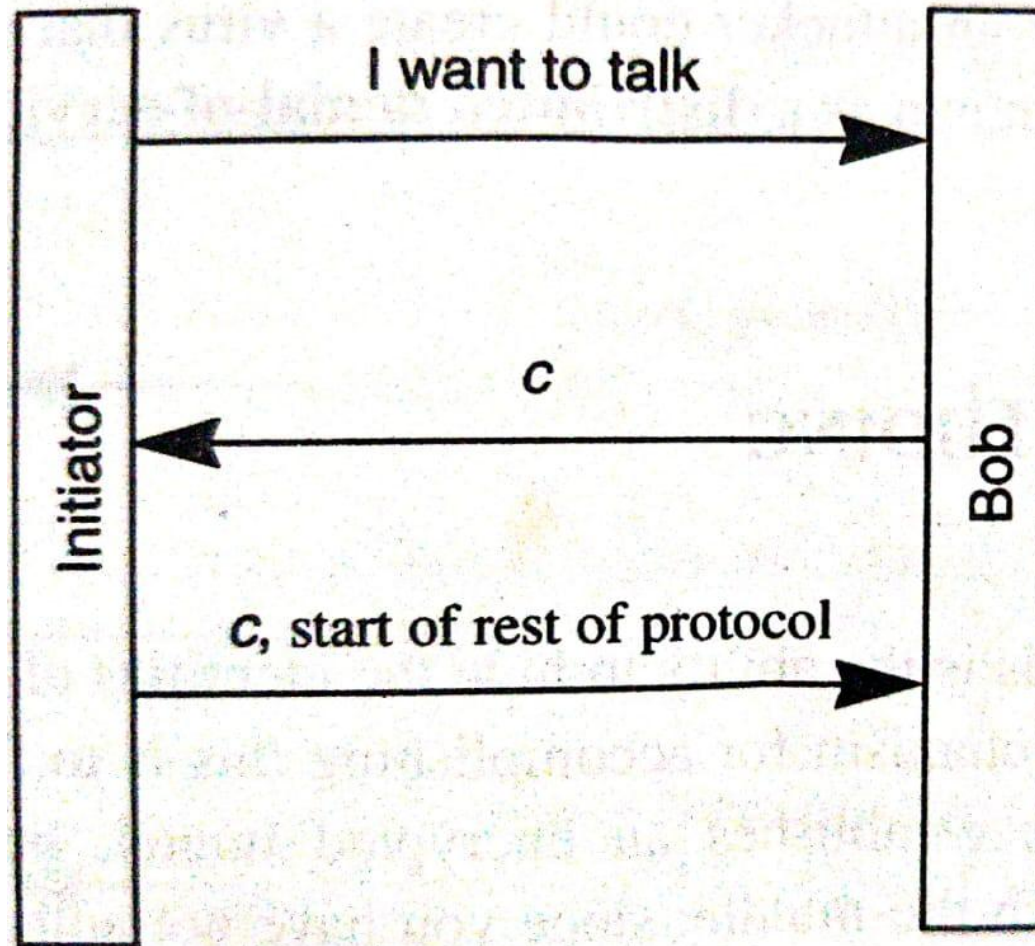
In TCP connection setup (3-way handshake):

- ✓ Client → Server: SYN
- ✓ Server → Client: SYN-ACK
- ✓ Client → Server: ACK

Attack idea:

- ✓ Attacker sends huge number of SYN requests
- ✓ Server allocates memory for each request
- ✓ Attacker never sends ACK
- ✓ Server's connection table fills → service unavailable

Solution: SYN Cookies:: SYN cookies prevent the server from allocating memory until the client proves it is genuine.



$c = \text{hash}(\text{IP address}, \text{secret})$

Does $c = \text{hash}(\text{IP address}, \text{secret})$?
If so, continue with protocol.

Protocol 16-3. Stateless cookie protocol

How Stateless Cookie Protocol Works?

1. Initiator (Client) Requests a Connection:
 - The initiator (client) sends a "I want to talk" request to Bob (server).
2. Bob Generates a Cookie (c):
 - Bob computes a stateless cookie c as: $c = \text{hash}(\text{IP address}, \text{secret})$
 - This cookie is unique for each client based on their IP address and a secret value known only to Bob.
 - The server does not store any session state yet, preventing unnecessary memory usage in case of an attack.
3. Bob Sends the Cookie Back to the Initiator:
 - The initiator receives c and must return it to proceed with the protocol.

4. Initiator Responds with the Cookie:

- The client sends c back to Bob, along with the start of the actual protocol (e.g., key exchange).

5. Bob Verifies the Cookie:

- Bob checks if the received c matches the expected value: $c = \text{hash}(\text{IP address}, \text{secret})$
- If the value is correct, Bob proceeds with the connection
- If the value is incorrect, the request is discarded, stopping malicious clients.

Computational Puzzles for Clogging Protection

- Computational puzzles ensure that clients must **perform some work** before establishing a session.
- This makes **DoS attacks expensive** while keeping legitimate users unaffected.
- **How Computational Puzzles Work**
 1. **Client sends a request** (e.g., start a WebRTC call).
 2. **Server responds with a puzzle challenge.**
 3. **Client must solve the puzzle** and return the correct answer.
 4. **If correct**, the server proceeds with the connection.

- Benefit: Attackers need massive computing power to generate fake requests at scale, making large-scale DoS attacks impractical
- Example: WebRTC Puzzle-Based Protection
 - Before setting up a WebRTC session, the server sends a mathematical puzzle (e.g., finding a hash value).
 - A real client solves it quickly, but attackers struggle to send thousands of valid solutions.
 - This reduces bot-driven DoS attacks without affecting real users.
 - Stops automated bots by making spam connections expensive. Prevents connection flooding in video calls or VoIP sessions.

Endpoint identifier hiding

- Endpoint Identifier Hiding is a security mechanism used to **conceal the identity (IP address, device information, or user details) of communication endpoints in a network.**
- Implemented in secure communication protocols to prevent attackers from tracking or targeting specific users or devices.
- Without identifier hiding, attackers can:
 - ✓ **Track users by analysing their IP addresses or device identifiers.**
 - ✓ **Launch targeted attacks** (e.g., DDoS, phishing) against specific users.
 - ✓ **Perform traffic analysis to infer communication patterns.**

Where is Endpoint Identifier Hiding Used?

- **VoIP & Real-Time Communication**

- ✓ **SIP (Session Initiation Protocol):** Some VoIP systems hide the caller's IP to prevent attackers from eavesdropping or tracking.
- ✓ **RTP (Real-time Transport Protocol):** Secure VoIP protocols use encryption and proxy servers to mask endpoint details.

- **VPNs & Anonymization Networks**

- ✓ **VPNs (Virtual Private Networks):** Hide user IP addresses by routing traffic through a secure server.
- ✓ **Tor Network:** Uses multiple relays to anonymize endpoints, making it difficult to track the original sender.

- **Mobile & Internet Protocols**

- ✓ **3G/4G/5G Networks:** Mobile networks assign temporary identifiers instead of revealing permanent subscriber details.

- ✓ **IPsec (Internet Protocol Security):** Can use ESP (Encapsulating Security Payload) to encrypt headers and hide endpoints.

- **Messaging & Secure Communications**

- ✓ **Signal, WhatsApp, and other encrypted messengers:** Use techniques like "Sealed Sender" to hide metadata about message senders.

- **How is Endpoint Identifier Hiding Implemented?**

- ✓ **Encryption** – Encrypting headers to hide endpoint details.
- ✓ **Proxy Servers** – Routing traffic through intermediate servers (e.g., VPNs, VoIP proxies).
- ✓ **Relay Networks** – Using multiple relays (like Tor) to anonymize traffic.
- ✓ **Temporary Identifiers** – Assigning random IDs instead of static addresses (e.g., mobile networks).

How It Enhances Real-Time Communication Security

- **Prevents IP Tracking** – Hides the real IP address of users in VoIP and messaging applications, preventing attackers from tracking their location or launching direct attacks.
- **Mitigates DoS & Clogging Attacks** – By masking endpoint identifiers, attackers cannot easily target a specific user's device or server to flood them with traffic.
- **Enhances Privacy** – Protects users in secure VoIP calls, messaging apps (like Signal/WhatsApp), and live streaming by preventing metadata leaks.
- **Defends Against Man-in-the-Middle (MITM) Attacks** – If endpoint identifiers are hidden, attackers cannot easily intercept and modify communication sessions.

Live Partner Reassurance

- Mechanisms that ensure **continuous authentication and trust** between communicating parties during a live session, such as a VoIP call, video conference, or secure chat.
- In real-time communication, there is always a risk of:
 - ✓ **Man-in-the-Middle (MITM) attacks** – An attacker intercepts or alters communication.
 - ✓ **Session Hijacking** – An unauthorized user takes over an ongoing session.
 - ✓ **Impersonation & Spoofing** – Attackers pretend to be a trusted contact.
- **Live Partner Reassurance helps mitigate these risks by continuously verifying that both parties remain authentic and secure throughout the session.**

How Does Live Partner Reassurance Work?

- LPR uses **real-time security checks** such as:
 - ✓ **End-to-End Encryption (E2EE)** – Ensures that only the intended participants can decrypt messages (used in **WhatsApp, Signal, Zoom, WebRTC**).
 - ✓ **Session Re-authentication** – Periodically verifies the participants' identities to prevent session hijacking.
 - ✓ **Trust Indicators** – Displays security indicators (such as a verified identity badge) to assure users they are talking to the correct person.
 - ✓ **Voice/Video Fingerprinting** – Uses **biometric analysis** or cryptographic hashing to detect impersonation attempts in VoIP calls.
 - ✓ **Real-Time Security Alerts** – Warns users if any unusual activity is detected, such as a new device joining the session.

Examples of Live Partner Reassurance in Action

- **Secure VoIP Calls (SIP/WebRTC)** – VoIP systems use encryption, call authentication, and secure handshake protocols to maintain trust.
- **Video Conferencing (Zoom, Teams, WebEx)** – These platforms use **participant verification**, waiting rooms, and security notifications to reassure users.
- **Corporate Secure Communication (VPN, TLS, IPsec)** – Ensures continuous secure encryption and prevents session takeovers.